

BACKEND DEVELOPER ASSIGNMENT

Mini LMS Module (CRM Integrated)

Target Tech Stack (Mandatory)

- **Backend Framework:** Node.js with TypeScript (Express or NestJS)
- **Database:** PostgreSQL/MongoDB
- **ORM / Query Builder:** Prisma / TypeORM / Knex

Objective

Evaluate the candidate's ability to design and develop a robust backend API for a simplified Learning Management System (LMS) as a module within a larger CRM ecosystem.

The system should demonstrate:

- Clean backend architecture
- Scalable relational data modeling
- API-first design suitable for multiple portals (Admin CRM, Student Portal, Mobile Apps)
- Proper authentication and authorization

User Roles

1. Admin (CRM User)

- Create and manage students
- Create and manage courses
- Assign courses to students
- View student-wise and course-wise progress reports

2. Student (Portal User)

- View assigned courses
- Access course videos
- Track video progress and resume from last position
- View personal learning progress

Core Functional Requirements

1. Student Management (Admin APIs)

- Create students (name, email, password, unique ID)
- Update student information
- List all students
- Get student by ID

2. Course and Content Management (Admin APIs)

- Create courses (name, description)
- Add video lessons (title, description, video source URL, duration)
- Update/delete courses and lessons
- List all courses with lessons

3. Course Assignment (Admin APIs)

- Assign courses to students
- Remove course assignments
- View assignments per student/course

4. Student Course Access (Student APIs)

- List assigned courses
- Get course details with lessons
- Access video lesson details

5. Video Progress Tracking (Student APIs)

- Update progress (video ID, timestamp, percentage, completion status)
- Get progress for specific video or entire course
- Mark video complete at 90% watched

6. Reporting and Analytics (Admin APIs)

- Student-wise progress (completion %, videos completed)
- Course-wise completion status
- Time spent and detailed metrics

API Design Requirements

Authentication:

- JWT-based authentication
- Separate login for Admin and Student
- Role-based access control (RBAC)

Example API Structure:

POST /api/auth/admin/login
POST /api/auth/student/login

POST /api/admin/students
GET /api/admin/students
POST /api/admin/courses
POST /api/admin/courses/:courseId/lessons
POST /api/admin/enrollments
GET /api/admin/reports/students/:studentId/progress

GET /api/student/courses
POST /api/student/progress
GET /api/student/courses/:courseId/progress

Response Format:

- Consistent JSON structure
- Proper HTTP status codes
- Error handling with meaningful messages
- Pagination for lists

Database Requirements

Schema Tables:

- users (admins and students)
- courses
- lessons
- enrollments
- video_progress

Requirements:

- Foreign key relationships
- Indexes on frequently queried fields
- Unique constraints
- Timestamps (created_at, updated_at)
- Migrations for schema management

Technical Expectations

Architecture:

- Controllers, Services, Repositories/DAL
- Middleware (auth, validation, error handling)

- TypeScript interfaces and types
- Input validation and sanitization

Security:

- Password hashing (bcrypt)
- JWT token management
- SQL injection prevention

Performance:

- Efficient queries
- Proper indexing
- Avoid N+1 problems

Bonus (Optional)

- API documentation (Swagger)
- Docker setup
- Unit/integration tests
- Logging middleware

Submission

- GitHub repo with clear README
- Setup instructions, environment variables
- API documentation
- Architecture decisions
- Database schema with ER diagram